Preguntas de examen PSyD

Daniel Báscones García

15 de octubre de 2026

Índice

1. Introducción y aviso **1**

2. Preguntas **1**

1. Introducción y aviso

En las siguientes páginas, se muestran **ejemplos** de preguntas de examen. Esto no quiere decir, ni que todas las preguntas vayan a salir de aquí (seguramente no), ni que sean refritos o adaptaciones de las mismas. Los contenidos que entran en examen incluyen tanto la teoría de los documentos teóricos, como la de las prácticas, y puede entrar cualquier cosa de aquellos siempre y cuando se haya dado en clase.

Así pues, esta colección sirve solo como referencia para entender la profundidad y dificultad que se puede esperar del examen, pero en ningún caso sustituye su resolución a estudiar la materia.

2. Preguntas

En construcción...

1. ¿Qué efecto produce en la compilación calificar un puntero a un registro con la palabra clave `volatile` en lenguaje C?

   Solución: *Al nombrarlo como `volatile`, le decimos al compilador que su valor podría cambiar por fuentes ajenas al programa (por ejemplo el propio periférico). Por tanto, no debería realizar optimizaciones basadas en suposiciones sobre su valor, leyéndolo siempre que lo necesite directamente desde memoria.*

2. Escriba una función en C `void gpio_set_mode(GPIO_TypeDef *GPIOx, uint32_t pin, uint32_t mode)` que configure de forma atómica a nivel de software el modo de un pin en el registro `MODER` (donde cada pin ocupa 2 bits). Asegúrese de limpiar primero la configuración previa mediante máscaras binarias antes de aplicar el nuevo modo

   Solución:

```
1 static inline void gpio_set_mode(GPIO_TypeDef *GPIOx, uint32_t pin,
      GPIO_Mode_t mode) {
2     uint32_t temp = GPIOx->MODER;
```

```
    temp &= ~(0b11U << (pin * 2));
    temp |= ((uint32_t)mode << (pin * 2));
    GPIOx->MODER = temp;
}
```

3. En el microcontrolador STM32F723, el controlador de interrupciones NVIC maneja registros vectorizados como `ISER` (Interrupt Set Enable Register), definidos dentro del `struct` del periférico como un array `volatile uint32_t ISER[8]` para cubrir hasta 256 líneas de interrupción. Explique matemáticamente cómo se traduce un número global de línea de interrupción `IRQn` (por ejemplo, 71) a un índice dentro del array `ISER[]` y a una posición de bit dentro de ese elemento de 32 bits.

   Solución: *Al ser cada elemento del array una palabra de 32 bits, las línea de interrupción se distribuyen en bloques contiguos de 32 líneas.*

   El índice del registro es la parte entera de la división por 32: $\lfloor N/32 \rfloor$. En operaciones de desplazamiento de bits equivale a $N \gg 5$. En nuestro ejemplo, $71 \gg 5 = 2$ (corresponde a `ISER[2]`).

   La posición del bit dentro del registro de 32 bits es el resto de la división: $N \pmod{32}$. En operaciones bit a bit equivale a N & $0x1F$ (N & 31).

4. ¿Qué sucede al intentar modificar los registros de control de un periférico (por ejemplo `GPIOA`) si no se ha activado previamente su reloj en el registro `RCC` correspondiente? (En este caso, `RCC_AHB1ENR`)

   Solución: *Como el periférico y sus registros no están recibiendo señal de reloj, no actualizarán sus valores y por tanto las escrituras habrán sido en vano.*

5. Si la frecuencia de reloj de entrada del periférico `USART6` es de 16 MHz y se requiere una velocidad de 115200 baudios ¿qué valor entero (aproximadamente) debe cargarse en el registro `USART6_BRR`?

   Solución: *Para reducir la velocidad de reloj de entrada a la objetivo (los baudios) debemos utilizar un divisor de reloj. En este caso, el valor que le damos está en el registro `BRR`. El valor v_{BRR} necesario será:*

$$v_{BRR} = \frac{f_{in}}{baudrate} = \frac{16 \cdot 10^6}{115200} \approx 139$$

6. El gestor de interrupciones externas `EXTI` tiene sólo unas pocas líneas (en concreto 16), muchas menos que la totalidad de pines de los que dispone el procesador (16 pines por puerto, en 10 puertos diferentes). ¿Cuántos pines puedo utilizar simultáneamente para interrumpir al procesador? ¿Qué restricciones existen?

   Solución: *`EXTI` multiplexa los pines de entrada, colocando todos los pines "0"(PA0, PB0, PC0,...) en la línea `EXTI0`, y así sucesivamente. Por tanto, el máximo de pines utilizables para interrumpir es de 16. La restricción existente es que nunca podremos utilizar dos pines con el mismo número (por ejemplo PA0 y PB0) a la vez, ya que comparten línea. Sí es posible sin embargo utilizar todos los pines de un puerto a la vez (PA0, PA1, PA2...) ya que cada uno va a una línea diferente.*

7. En el código de inicialización del procesador, se definen todas las rutinas de tratamiento de interrupción. Por defecto, todas ellas apuntan a la misma función llamada `Default_Handler`. ¿Cómo se consigue cambiar esto al redefinir una ISR manualmente? ¿Por qué no se sigue apuntando a `Default_Handler`?

Solución: *Aunque por defecto se apunte a* `Default_Handler`*, su puntero está definido como* `.weak`*. Esto quiere decir que cualquier redefinición del mismo ganará prioridad sobre la original. Por tanto, al redefinirlo, conseguimos que el enlazador apunte a la nueva función en lugar de la antigua.*